

```

import os
import json
import glob

# PDF o'qish uchun: pip install PyPDF2
try:
    import PyPDF2
    PDF_SUPPORT = True
except ImportError:
    PDF_SUPPORT = False

PDF_FOLDER = "pdfs" # PDF kitoblar shu papkada bo'ladi
CACHE_FILE = "pdf_cache.json"

class PDFLoader:
    def __init__(self):
        self.pdf_texts = {}
        self._load_pdfs()

    def _load_pdfs(self):
        """Barcha PDF larni yuklash va cache qilish"""
        # Cache mavjud bo'lsa, undan o'qish
        if os.path.exists(CACHE_FILE):
            try:
                with open(CACHE_FILE, "r", encoding="utf-8") as f:
                    self.pdf_texts = json.load(f)
                print(f"✅ Cache yuklandi: {len(self.pdf_texts)} ta kitob")
                return
            except:
                pass

        # PDF papka yo'q bo'lsa yaratish
        if not os.path.exists(PDF_FOLDER):
            os.makedirs(PDF_FOLDER)
            print(f"📁 '{PDF_FOLDER}' papkasi yaratildi. PDF kitoblarni shu yerga")
            return

        if not PDF_SUPPORT:
            print(f"⚠️ PyPDF2 o'rnatilmagan. 'pip install PyPDF2' bajaring.")
            return

        # PDF larni o'qish
        pdf_files = glob.glob(os.path.join(PDF_FOLDER, "*.pdf"))

```

```

print(f" {len(pdf_files)} ta PDF topildi...")

for pdf_path in pdf_files:
    filename = os.path.basename(pdf_path)
    try:
        text = self._extract_pdf_text(pdf_path)
        self.pdf_texts[filename] = text
        print(f" O'qildi: {filename}")
    except Exception as e:
        print(f" Xato {filename}: {e}")

# Cache saqlash
if self.pdf_texts:
    with open(CACHE_FILE, "w", encoding="utf-8") as f:
        json.dump(self.pdf_texts, f, ensure_ascii=False)
    print(f" Cache saqlandi")

def _extract_pdf_text(self, pdf_path: str) -> str:
    """PDF dan matn olish"""
    text = ""
    with open(pdf_path, "rb") as f:
        reader = PyPDF2.PdfReader(f)
        for page in reader.pages:
            text += page.extract_text() + "\n"
    return text

def search_in_pdfs(self, query: str, max_chars: int = 3000) -> str:
    """Savolga mos kontekst qidirish"""
    if not self.pdf_texts:
        return ""

    query_lower = query.lower()
    query_words = query_lower.split()

    # Har bir PDF dan mos qismlarni topish
    results = []
    for filename, text in self.pdf_texts.items():
        # Sinf nomini fayl nomidan aniqlash (masalan: "biologiya_7.pdf")
        grade_info = ""
        for g in ["5", "6", "7", "8", "9", "10", "11"]:
            if g in filename:
                grade_info = f"{g}-sinf"
                break

        # Paragraflarni ajratish
        paragraphs = text.split("\n")
        for para in paragraphs:

```

```

        if len(para.strip()) < 20:
            continue
        para_lower = para.lower()
        # Mos so'zlarni hisoblash
        score = sum(1 for word in query_words if word in para_lower)
        if score > 0:
            results.append((score, grade_info, para.strip()))

    if not results:
        return ""

    # Eng mos natijalarni tanlash
    results.sort(reverse=True)
    top_results = results[:5]

    context = "📖 Darslik ma'lumotlari:\n\n"
    for score, grade, para in top_results:
        prefix = f"[{grade}] " if grade else ""
        context += f"{prefix}{para}\n\n"

    # Uzunlikni cheklash
    if len(context) > max_chars:
        context = context[:max_chars] + "..."

    return context

def reload_pdfs(self):
    """PDF larni qayta yuklash (yangi kitob qo'shilganda)"""
    if os.path.exists(CACHE_FILE):
        os.remove(CACHE_FILE)
    self.pdf_texts = {}
    self._load_pdfs()

```